

Java Garbage Collection & Memory Issues – Detailed Revision Notes

Does Java Process Memory Go Beyond -Xmx?

Yes. The -Xmx parameter only defines the maximum Java heap size. The total process memory used by the JVM is the sum of several components: - Heap memory (defined by -Xmx) - Metaspace/Class metadata storage - Thread stacks (each thread consumes native stack memory) - Direct memory allocations - Other native memory requirements (JNI, OS allocations) This is why monitoring tools like 'top' or 'ps' may show memory usage greater than -Xmx.

Out Of Memory Error Types

Java can throw different kinds of OutOfMemoryError depending on what part of memory is exhausted: 1. **Java heap space** – Objects cannot be allocated because the heap is full and GC cannot free space. 2. **GC overhead limit exceeded** – JVM spends >98% of its time in garbage collection but recovers <2% of heap. 3. **Requested array size exceeds VM limit** – Attempt to allocate an array bigger than Integer.MAX_VALUE. 4. **Metaspace** – Class metadata space (Java 8+), runs out due to excessive class loading. 5. **PermGen space** – Pre-Java 8 metadata space for classes. 6. **Direct buffer memory** – Off-heap memory for ByteBuffer.allocateDirect(). 7. **Unable to create new native thread** – OS cannot create more threads due to limits or memory exhaustion.

Common Causes of OOM

- **Memory leaks**: Holding references to objects that are no longer needed. - **Very large data structures**: Storing too much data in memory at once. - **Unbounded class loading**: Dynamic class loading without proper unloading. - **Excessive threads**: Too many active threads consuming stack space. - **High direct memory usage**: Excessive use of off-heap buffers.

GC Behavior Patterns

1. **Healthy Application** – Heap usage rises during allocation, then drops sharply after GC. Pattern is cyclic and stable. 2. **Acute Memory Leak** – Heap grows rapidly without dropping significantly after GC; frequent Full GCs occur. 3. **Gradual Memory Leak** – Heap grows slowly over time; GC becomes more frequent before eventual OOM.

Key Diagnostics

1. **GC Logs** – Use tools like GCeasy to visualize GC behavior and memory trends. 2. **Heap Dumps** – Capture using '-XX:+HeapDumpOnOutOfMemoryError' or tools like jmap, VisualVM. Analyze with Eclipse MAT or HeapHero to identify largest objects and reference chains. 3. **Thread Dumps** – Capture with 'jstack' or 'kill -3'. Useful for detecting deadlocks, blocked threads, or high CPU usage patterns.

Special OOM Cases

- **GC Overhead Limit Exceeded**: Typically due to severe leaks or massive data processing. Solution: fix leaks or increase heap. - **Requested Array Size Exceeds VM Limit**: Avoid reading extremely large files into one array. - **Metaspace OOM**: Limit via `-XX:MaxMetaspaceSize` and reduce dynamic class generation. - **PermGen Space**: Only in pre-Java 8; avoid excessive classloading. - **Unable to Create New Native Threads**: Reduce thread count or increase OS limits. - **Direct Buffer Memory**: Limit via `-XX:MaxDirectMemorySize`. - **Kill Process or Sacrifice Child**: Linux OOM killer terminates processes to save system memory.

Real Case Studies

1. **Major US Financial Institution** – Application slowdown traced to threads blocked on I/O. Solved by identifying bottleneck through thread dump analysis. 2. **AWS EBS Service HTTP 502 Errors** – Native memory exhaustion due to unbounded direct buffer allocations; fixed by capping direct memory usage.